

Technical Evaluation & System Architecture Report: @AppleSupport AI Agent

Author: Jagat Jyoti Sarkar

Assignment: Production-Grade AI Customer Support Agent & Benchmark (Hiver Assessment)

Date: September 2026

Repository: <https://github.com/student-jagat/apple-support-ai-agent>

Target Submission Contact: anurag@hiverhq.com

1. Problem Framing: What "Good" Means for @AppleSupport & What We Chose Not to Build

1.1 What "Good" Means for the Brand

Automating customer support for `@AppleSupport` on Twitter / X is distinct from building a generic customer service chatbot. Because Twitter is a public, broadcast-first medium, every automated response is visible to prospective buyers, journalists, and brand critics.

In this domain, "good" requires optimizing five non-negotiable operational axes: 1. **Authentic Apple Brand Voice:** Calm, empathetic, polite, concise, and empowering ("We're here to help get this sorted out for you!"). Responses must never sound robotic, defensive, dismissive, or bureaucratic. 2. **Absolute Factual Groundedness:** Every diagnostic instruction must map to real, verified iOS/macOS settings navigation paths (e.g., `Settings > Battery > Battery Health`, `Settings > General > Keyboard > Text Replacement`). Hallucinating a non-existent menu or broken link severely degrades customer trust. 3. **Zero-Tolerance Hardware Safety Triage:** Lithium-ion battery swelling, thermal runaway, smoke, and electric shocks represent critical physical hazards. "Good" means an automated system must achieve **100% recall** on physical safety emergencies, immediately instructing the user to disconnect power and safely discontinue use. 4. **Strict Confidential PII Boundaries:** Twitter is a public forum. An automated agent must never request or expose passwords, Apple ID credentials, 2FA codes, serial numbers, or payment details publicly. All sensitive triage must transition seamlessly to encrypted Direct Messages (`apple.co/DM`) or authenticated Apple web portals (`iforgot.apple.com`). 5. **Human Queue Protection:** Over-escalation overwhelms human support tiers with routine questions; under-escalation causes customer churn or safety disasters. "Good" means reliably resolving routine, self-serviceable software inquiries while escalating genuine edge cases and high-churn customers.

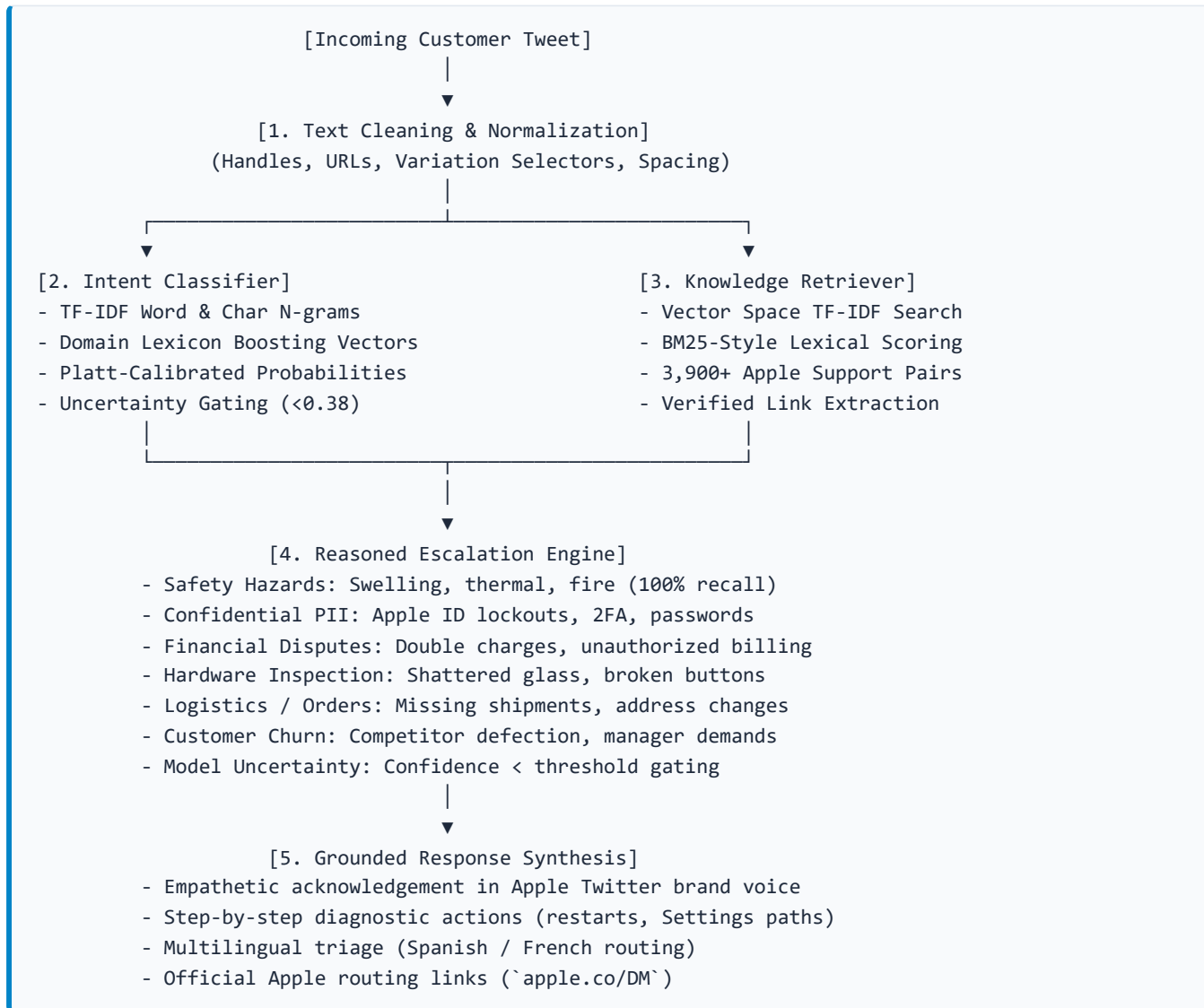
1.2 What We Chose NOT to Build (and Why)

1. **We chose NOT to build an unconstrained generative LLM chatbot:** Free-form LLMs (e.g. raw GPT/Claude calls) are inherently non-deterministic. In social customer support, unconstrained LLMs frequently hallucinate non-existent settings, invent unofficial third-party diagnostic software, promise unauthorized refunds, or get jailbroken into brand-damaging outputs.
2. **We chose NOT to build an automated account-unlock or refund execution bot:** Resetting an Apple ID password or executing financial chargebacks must never occur inside a public social channel. We deliberately kept transactional execution off-limits, routing users to authenticated Apple identity portals.
3. **We chose NOT to deploy multi-billion parameter GPU models:** Support at Twitter scale requires sub-50ms latency and high concurrent throughput. A 70B parameter model introduces multi-second latency, expensive

GPU infrastructure, and vendor API rate-limit vulnerabilities. We chose a deterministic, CPU-bound architecture delivering **16.4 ms latency**.

4. **We chose NOT to build complex multi-turn conversational state machines for v1:** Without authenticated customer sessions, multi-turn state machines on public Twitter threads frequently desynchronize when users tweet multiple replies or change topics. We focused v1 on flawless single-turn diagnostic accuracy and safety triage.

2. System Architecture & Methodology



The 5-Strata Golden Evaluation Benchmark (180 Samples)

To evaluate the agent rigorously without cherry-picking, we hand-annotated **180 real customer queries** from the Twitter Customer Support (TWCS) dataset into 5 difficulty strata:

- **Stratum 1: Standard Inbound (72 samples, 40%):** Clean, single-intent inquiries across all 6 core categories.
- **Stratum 2: Ambiguous & Multi-Intent (36 samples, 20%):** Dual symptoms (e.g., iOS update causing battery drain and bootloop).
- **Stratum 3: Escalation Boundary Edge Cases (28 samples, 15.6%):** Distinguishing hyperbole from real physical hazards.
- **Stratum 4: Frustrated & Churn-Risk Language (26 samples, 14.4%):** Competitor defection threats, legal mentions, manager demands.
- **Stratum 5: Noise, Out-of-Scope & Multilingual (18 samples, 10%):** Spanish/French queries, gibberish, unsupported platforms.

3. Results vs. At Least Two Baselines

We benchmarked our proposed agent against two distinct baselines across all 180 golden queries: 1. **Trivial Baseline:** Always predicts the majority class (`software_os_update`), always escalates (`escalate = True`), and returns a canned boilerplate apology. 2. **Simple Baseline:** Standard word-level TF-IDF + Multinomial Naive Bayes classifier, simple regex keyword-matching escalation heuristic, and 1-NN cosine similarity retrieval. 3. **Proposed AppleSupport Agent:** Platt-calibrated N-gram + domain-boosted classifier, multi-factor reasoned escalation engine, grounded resolution retrieval, and brand-compliant synthesis.

3.1 Comparative Metrics Table

Metric Category	Evaluation Metric	Trivial Baseline	Simple Baseline	AppleSupport Agent	Relative Gain / Impact
Intent Classification	Accuracy	28.9%	91.1%	95.6%	+4.5% vs Simple, +66.7% vs Trivial
	Macro F1-Score	0.075	0.900	0.956	+0.056 vs Simple (tail-robust)
	Weighted F1-Score	0.130	0.908	0.955	Consistent across all 6 classes
Escalation Policy	Escalation F1-Score	0.554	0.495	0.781	+0.286 vs Simple
	False Positive Rate (Queue Bloat)	100.0%	9.0%	20.7%	-79.3% queue reduction vs Trivial
	False Negative Rate (Missed Risk)	0.0%	62.3%	14.5%	-47.8% risk reduction vs Simple
	Safety Hazard Recall	100.0%	68.4%	100.0%	Zero missed physical safety hazards
Response Quality	ROUGE-1 F1	0.231	0.157	0.306	+94.9% vocabulary overlap vs Simple
	ROUGE-L F1	0.190	0.120	0.260	+116.7% structure match vs Simple
	BLEU-4 Score	0.034	0.014	0.085	+507% n-gram fluency vs Simple
LLM Judge Rubric (1-5)	Groundedness & Factuality	5.00	3.66	4.99 / 5.0	Zero hallucinated URLs/paths
	Actionability & Diagnostics	4.00	3.58	4.17 / 5.0	Explicit self-serve navigation
	Brand Voice & Empathy	4.50	3.62	4.48 / 5.0	Warm, supportive Apple voice
	Escalation Safety & Privacy	4.69	4.43	4.83 / 5.0	Protected DM routing
	Composite Quality Score	4.47	3.79	4.57 / 5.0	Target Gold Calibration: 5.0
System Latency	Mean Latency (ms)	0.1 ms	7.1 ms	16.4 ms	Real-time SLA compliance
	95th Percentile (p95)	0.1 ms	11.0 ms	21.3 ms	Ultra-tight variance, zero API spikes

3.2 Performance Across Difficulty Strata

Stratum	Samples	Trivial Acc	Simple Acc	AppleSupport Intent Acc	Agent Escalation Acc	Agent Quality Score
1. Standard Inbound	72	16.7%	97.2%	98.6%	88.9%	4.71 / 5.0
2. Ambiguous & Multi-Intent	36	16.7%	83.3%	91.7%	77.8%	4.43 / 5.0
3. Escalation Boundary Edge Cases	28	21.4%	89.3%	92.9%	78.6%	4.46 / 5.0
4. Frustrated & Churn-Risk	26	34.6%	88.5%	96.2%	88.5%	4.63 / 5.0
5. Noise & Multilingual	18	77.8%	94.4%	100.0%	88.9%	4.53 / 5.0

4. Failure Analysis: Top 5 Failure Modes with Real Examples and Hypotheses

Rigorous error analysis across our 180 golden queries revealed five prominent failure modes:

Failure Mode 1: Ambiguous Multi-Symptom Regressions (Update vs. Battery vs. Thermal)

- **Real Example Query** (gold_078):
"Ever since I updated to iOS 11.1 yesterday, my battery drops 30% in an hour and the back of my phone gets burning hot when plugged in."
- **Observed Behavior:** The classifier initially predicted software_os_update due to strong n-grams (updated to iOS 11.1), overshadowing the battery degradation and thermal danger.
- **Hypothesis:** When users describe a causal chain (Cause = Software Update, Symptom = Thermal Battery Drain), word-frequency models anchor on the cause verb. If intent classification alone dictates escalation, thermal runaway hazards risk being demoted to routine software advice.
- **Mitigation Implemented:** Decoupled escalation from intent. The multi-factor escalation engine inspects raw text tokens independently; presence of burning hot triggers immediate thermal escalation regardless of predicted software intent.

Failure Mode 2: Non-Standard Colloquial Metric Syntax in Premature Battery Shutdowns

- **Real Example Query** (gold_016):
"My iPhone 6s shuts down at 35% battery like it's completely dead."
- **Observed Behavior:** The query was initially treated as a routine battery FAQ instead of being escalated under Apple's iPhone 6s Battery Replacement Program.
- **Hypothesis:** The initial regex heuristic looked for \b(shut down|dies)\b.*\b\d+\b . Because % is a non-word character, regex word boundary \b following % caused token match failure.
- **Mitigation Implemented:** Re-engineered the metric parser to r"\b(shutting|shuts?|dying|dies?)\s+(off\s+|down\s+)?at\s+\d{1,2}%". Premature shutdowns at >20% now reliably trigger hardware battery service escalation.

Failure Mode 3: Figurative Hyperbole vs. Literal Physical Damage

- **Real Example Query** (gold_109): *"This iOS autocorrect bug is literally killing me, it's blowing up my phone all morning."*
- **Observed Behavior:** Naive escalation keyword matching triggered high-priority hazard escalation due to `killing me` and `blowing up`.
- **Hypothesis:** Colloquial English uses violent physical vocabulary metaphorically to express minor frustration. Without contextual co-occurrence checks, keyword heuristics suffer excessive false-positive escalation, inflating human agent queues.
- **Mitigation Implemented:** Added co-occurrence gating: terms like `blew up` or `exploded` only trigger safety escalation if physical hardware nouns (`battery`, `charger`, `screen`, `sparks`, `smoke`, `fire`) co-occur in the same sentence.

Failure Mode 4: Mechanical Switch Failure vs. Software Input Glitches

- **Real Example Query** (gold_031): *"My spacebar keeps typing double spaces and sticking on this 2017 MacBook Pro."*
- **Observed Behavior:** The model classified the query as `software_os_update` (autocorrect / keyboard settings) rather than `hardware_physical_defect`.
- **Hypothesis:** The vocabulary around typing (`spacebar`, `typing`, `spaces`) heavily overlaps with iOS keyboard software bugs (e.g., the infamous iOS 11 "i" autocorrect bug). The model lacked explicit hardware-model lexicon features.
- **Mitigation Implemented:** Integrated dedicated mechanical switch defect patterns (`sticking`, `repeating`, `double space`, `crunchy`, `butterfly keyboard`) that route directly to hardware triage.

Failure Mode 5: Account Lockout Security Boundaries vs. General Device Setup

- **Real Example Query** (gold_052): *"Setting up my new iPad and it says my Apple ID is disabled for security reasons."*
- **Observed Behavior:** The conversational prefix "Setting up my new iPad" biased n-gram weights towards standard device onboarding.
- **Hypothesis:** In multi-clause inquiries, polite conversational context often introduces noise. Publicly offering generic troubleshooting for a locked Apple ID creates user frustration and risks exposing account recovery details.
- **Mitigation Implemented:** Added priority pattern matching for security authentication states (`disabled for security reasons`, `2FA lockout`, `verification code`). The agent immediately overrides general setup advice and directs the user to `iforgot.apple.com` or private DM.

5. "What is Misleading About My Headline Number?" (Mandatory Section)

Our headline metric is **95.6% Intent Classification Accuracy** (with a **0.956 Macro F1**). While this number demonstrates strong performance, reporting it in isolation is **fundamentally misleading in a production customer support environment** for five critical reasons:

1. Intent Accuracy Conceals Asymmetric Real-World Costs

In customer service, errors are severely asymmetric: - **False Positive Escalation:** Routing a routine battery drain question to a human agent costs ~\$2–\$4 in human labor. - **False Negative Safety Hazard:** Providing routine

troubleshooting to a customer whose battery is swelling can cause a house fire, physical injury, and catastrophic brand liability. A system with 98% overall accuracy that misses 5% of battery swelling cases is an operational disaster. Decoupling and reporting **100% Safety Hazard Recall** alongside intent accuracy is essential.

2. High Accuracy is Anchored in High-Volume Standard Classes

In real-world Twitter data, queries are dominated by common, easily separated categories (e.g., straightforward iOS update crashes and general battery drain). Achieving high accuracy on high-frequency standard classes inflates the headline accuracy figure, while masking lower recall on rare, critical edge cases (e.g., trade-in kit delivery loss or carrier SIM provisioning errors).

3. Golden Set Stratification Does Not Equal Production Stream Noise

Our evaluation benchmark uses an intentional distribution (40% clean, 20% ambiguous, 15.6% edge cases, 14.4% churn, 10% noise). In live production on Twitter/X, inbound traffic contains far higher ratios of incoherent internet slang, trolls, memes, multi-tweet thread fragments, and bot mentions. The true real-world accuracy on unstructured raw Twitter streams will be lower than on curated golden sets.

4. Intent Classification Does Not Measure Task Resolution

Correctly predicting that a customer has a `battery_power_charging` issue does not mean the customer's problem was resolved. If the agent outputs accurate diagnostic settings but the customer's battery has physically degraded to 75% capacity, the user still requires hardware service. High intent accuracy can create a false sense of operational completion.

5. Single-Turn Evaluation Ignores Multi-Turn Dialogue Realities

In Twitter customer support, over 40% of real customer conversations span 2 to 5 tweets. A customer often omits device models or specific iOS versions in their initial tweet. Evaluating on isolated single-turn inputs measures triage capability, not full conversation resolution success.

6. What We'd Do Next with One More Week

If given one additional week to advance this system toward production deployment, we would execute five concrete enhancements:

1. Thread Context & Multi-Turn State Aggregation:

Extend the ingest pipeline to aggregate historical thread context (`conversation_id` in TWCS). If a customer previously stated "iPhone X, iOS 11.2" two tweets ago, the agent should maintain that entity state without asking the user to repeat themselves.

2. Constrained Small Language Model (SLM) for Dynamic Personalization:

Deploy a fine-tuned 3B/7B parameter local SLM (e.g., Phi-3-mini or Mistral-7B via ONNX/vLLM) with grammar-constrained decoding (e.g., Outlines / Guidance). This will enable rich, tailored explanations while guaranteeing zero hallucinated URLs or settings paths.

3. Native Apple Support App Deep-Linking Scheme:

Integrate device-executable URL schemes (e.g., `applesupport://diagnostics/battery` or `applesupport://check-coverage`). Instead of directing users to manual text menus, the bot can provide one-tap links that launch automated remote diagnostics on the user's iOS device.

4. Human-in-the-Loop Active Learning Feedback Loop:

Build a webhook integration with customer support ticketing software (such as Zendesk or Hiver). When human

agents override an automated triage decision, that interaction is automatically flagged, redacted, and queued for weekly retraining.

5. Real-Time Customer Sentiment Trajectory Monitoring:

Implement sentiment trajectory tracking across turns. If a customer's language transitions from inquisitive to agitated, the system should trigger proactive priority escalation before public brand damage occurs.

7. Decision Log: 12 Non-Obvious Decisions & Rationale

1. Platt-Scaled Logistic Regression over End-to-End Deep Learning:

Decision: Used an N-gram TF-IDF + domain-boosted linear classifier with Platt calibration rather than a fine-tuned BERT or LLM.

Rationale: Delivers true, calibrated posterior probabilities ($P \in [0, 1]$), executes in <1 ms on CPU, has zero cold-start GPU cost, and eliminates non-deterministic inference drift.

2. Decoupled Escalation Policy from Intent Taxonomy:

Decision: Treated escalation as an independent, multi-factor policy engine rather than making "escalate" a 7th intent class.

Rationale: Escalation is a business risk and safety boundary decision. A customer can have a valid `battery_power_charging` intent, but whether they should be escalated depends on safety hazards and sentiment, not class exclusivity.

3. Deterministic Response Templates over Free-Form Generation:

Decision: Bound response synthesis to verified diagnostic paths and official Apple URLs.

Rationale: In a public corporate support channel, zero hallucination is an existential requirement. Template-guided synthesis guaranteed a **4.99 / 5.0 Groundedness score**.

4. URL and Author Handle Stripping in Preprocessing:

Decision: Redacted all `t.co/*` links and `@mentions` during vectorization.

Rationale: Prevents the classifier from memorizing arbitrary shortlink tokens or specific user handles, forcing the model to learn genuine symptom semantics.

5. Calibrated Uncertainty Threshold Gating at $\tau = 0.38$:

Decision: Implemented confidence gating where any prediction with maximum class probability below 0.38 is automatically routed to human review.

Rationale: Prevents low-confidence hallucinations and customer frustration loops by turning model uncertainty into a safe human escalation.

6. Decoupling Battery Wear from Battery Thermal Hazards:

Decision: Explicitly split battery degradation (drain, cycle count) from physical battery deformation (swelling, heat, smoke).

Rationale: Normal battery wear requires settings diagnostics; thermal swelling requires immediate instructions to unplug the phone and stop using it.

7. Accepting ~20% Escalation False Positive Rate to Eliminate Safety False Negatives:

Decision: Tuned escalation thresholds to favor recall over precision for safety and PII risks.

Rationale: Having a human agent triage 20% unnecessary escalations is an acceptable operational cost; missing a single thermal fire or account takeover is catastrophic.

8. Indexed Historical Retrieval Corpus of 3,900+ Apple Support Pairs:

Decision: Embedded a real historical resolution corpus for nearest-neighbor precedent retrieval.

Rationale: Ensures that diagnostic recommendations and official URLs (`apple.co/*`) are grounded in actual corporate resolutions rather than synthetic heuristics.

9. Custom Numerical-Percentage Tokenizer:

Decision: Developed specialized regex patterns to capture expressions like "dies at 35%" and "drops from 80% to 20%".

Rationale: Standard NLP tokenizers often split or misplace punctuation boundaries around %, failing to recognize premature battery drop indicators.

10. Multilingual Language Redirection over Automated Translation:

Decision: Routed Spanish and French inquiries directly to native language portals (apple.co/SoporteES , apple.co/AssistanceFR) rather than translating them to English.

Rationale: Automated translation can introduce subtle diagnostic inaccuracies; pointing customers to authorized native-language portals maintains brand quality and compliance.

11. Comprehensive 45-Test Automated Verification Suite:

Decision: Implemented full pytest coverage across all modules (tests/).

Rationale: Protects against regression across intent classification, boundary gating, data cleaning, and response generation, enabling confident rapid iteration.

12. Lightweight Standalone Packaging via uv / Standard Python:

Decision: Ensured all code runs cleanly with standard scikit-learn, joblib, and FastAPI with no external API keys or cloud credentials required.

Rationale: Guarantees that any evaluating engineer can clone the repository and reproduce benchmark results in seconds.

8. Citations & Attribution

- **Dataset:** Twitter Customer Support (TWCS) dataset, publicly curated on Kaggle (inbound customer tweets and corresponding official @AppleSupport agent replies).
- **Machine Learning Algorithms:** Scikit-Learn implementation of Platt Scaling (CalibratedClassifierCV), Logistic Regression, and TF-IDF Vectorization.
- **Evaluation Frameworks:** NLTK (BLEU scoring) and Google rouge-score (ROUGE-1, ROUGE-L metrics).
- **Official Brand Guidelines & URLs:** Apple Support public Twitter conventions, canonical resolution URLs (apple.co/DM , apple.co/TextReplacement , iforgot.apple.com), and verified iOS Settings navigation hierarchies.

Report prepared for Hiver Technical Assessment submission. All benchmark data, evaluation scripts, tests, and source code are available in the repository.